Sampling for Region-Aggregated Spatial Scan Statistics

Foad Namjoo
University of Utah
foad.namjoo@utah.edu

Drew McClelland
Taptap Send
drewmac6191@gmail.com

Michael Matheny
Meta
michaelmathen@gmail.com

Jeff M. Phillips
University of Utah
jeffp@cs.utah.edu

Accepted at ACM SIGSPATIAL 2026.

Abstract

Anomaly detection in geospatial data is a crucial tool in geographic information science (GIS), with applications ranging from national security to public-health surveillance to the study of societal disparities. This work focuses on spatial scan statistics and addresses a key mismatch: spatial counts are typically aggregated into predefined regions (census tracts, zip codes, counties), whereas the most efficient scan algorithms operate on spatial point data. The standard remedy—collapsing each region to its centroid, as in widely used tools such as SaTScan—is convenient but, as we show, discards the region's spatial extent and causes a significant loss in statistical power. To resolve this, we propose a simple yet scalable fix: replace each spatial region with 20–50 points sampled uniformly from its geometry, and divide the region's measured and baseline counts evenly among them. This approach improves statistical power while maintaining computational tractability. A convergence analysis explains why so few samples per region suffice. We recommend this sampling-based conversion as the default way to apply point-based spatial scan statistics to region-aggregated data for anomaly detection. [1]

1 Introduction

Scan statistics (Kulldorff, 1997; Xie et al., 2022; Abolhassani and Prates, 2021; Glaz and Koutras, 2024) are a fundamental tool in spatial data analysis. They aim to identify spatial regions where a measured value is significantly different from what would be expected based on a baseline distribution. The statistic evaluates this by "scanning" all candidate spatial regions to detect the most anomalous ones. There are many variants of spatial scan statistics (Neill and Moore, 2004; Neill et al., 2006; Patil and Taillie, 2004; Kulldorff et al., 2006; Han et al., 2019), and they have been successfully used in various applications such as detecting elevated breast cancer rates (Kulldorff et al., 2006), monitoring emerging public health threats (Nobles et al., 2022; Desjardins et al., 2020), and analyzing spatial crime patterns (Shiode, 2011). These tools are central to anomaly detection workflows in GIS and public health surveillance.

Scan statistics can be applied to spatial data provided in two formats. The first format represents data as a set of geolocated points $X \subset \mathbb{R}^2$; such as home addresses or GPS-tracked locations. The most common software for these tasks operates on an input of this form, such as SaTScan (Kulldorff, 2022). The second common format aggregates the data into predefined regions (e.g., census tracts, zip codes,

[1]Thanks to NSF CCF-2115677, IIS-1816149, and IIS-2311954.

counties) (Xie et al., 2022; Tango and Takahashi, 2005). The baseline and measured values can then be accumulated for each region. These aggregations may be necessary due to privacy constraints (an individual's data can be less easily identified if it has been region aggregated) or because data are only available at a coarse spatial resolution.

To apply point-based algorithms to region-aggregated data, it is common practice to convert each region into a single point using its centroid. Although computationally convenient, this modeling choice introduces approximation errors and can significantly reduce the *statistical power*—the ability to reliably detect a true anomaly. We make this precise in Section 4. In particular, it does not capture the spatial diversity and extent of the underlying region, leading to weaker anomaly detection.

This paper addresses the above limitation and proposes a robust yet scalable alternative. *The proposed solution is simple:* ***instead of representing a region by one centroid point, we sample multiple points (typically 20 to 50) in each region and divide the region's measured and baseline counts uniformly across those points.*** We show that this approach significantly increases the statistical power of these methods under planted region experiments and demonstrate its scalability on datasets at local, state, and national levels. [2] By bridging region-aggregated data with efficient point-based algorithms, our method improves anomaly detection accuracy and strengthens spatial decision-making reliability.

2 Preliminaries

This section reviews both the classical point-based formulation and the challenges that arise when extending it to region-based settings.

Point-Based Spatial Scan Statistics In the point-based setting, spatial scan statistics operate on a set of spatial points $X \subset \mathbb{R}^2$, where each point $x \in X$ has an associated baseline value $b(x)$, which represents the expected background level of the measured phenomenon at location x (e.g., population or expected case count) and a measured value $m(x)$ representing the observed quantity of interest (e.g., people with cancer). The goal is to identify regions where the measured values deviate from the baseline, using a family of geometric shapes $\mathcal{S}$ and a cost function ϕ.

The family $\mathcal{S}$ is typically defined by a geometric shape. Common choices are disks (interiors of circles) or (axis-aligned) rectangles. Each shape $S \in \mathcal{S}$ induces a subset $Y = X \cap S$ of points, and the scan statistic evaluates each induced subset Y using the anomaly score function $\phi(Y)$.

A widely used cost function $\phi(Y)$, is derived under a Poisson model (Kulldorff, 1997) for the distribution of measured values $m(x)$ given a baseline value $b(x)$. An anomalous subset $Y \subset X$ is one that would be well modeled by a different (often larger) rate of measured values than $X \setminus Y$. The final anomaly score $\phi(Y)$ is derived as the log-likelihood ratio between the best model with a different rate for Y and $X \setminus Y$ and for the best model where the rate is the same for all X:

$$\phi(Y) = m(Y) \log \frac{m(Y)}{b(Y)} + (1 - m(Y)) \log \frac{1 - m(Y)}{1 - b(Y)},$$

where, $m(Y) = \frac{1}{M} \sum_{x \in Y} m(x)$ and $b(Y) = \frac{1}{B} \sum_{x \in Y} b(x)$, where $M = \sum_{x \in X} m(x)$ and $B = \sum_{x \in X} b(x)$. Although other cost functions exist (Kulldorff, 2022; Agarwal et al., 2006a) (e.g., under Bernoulli or Gaussian assumptions), we adopt the Poisson model due to its widespread use and compatibility with both point and region-based inputs.

The final step of spatial scan statistics is to identify the shape S that (approximately) maximizes $\phi(X \cap S)$. This defines the spatial region that is most anomalous, and the statistic itself is

$$\Phi_{\mathcal{S}}(X, m, b) = \max_{S \in \mathcal{S}} \phi(X \cap S).$$

While there are infinite number of geometric shapes that we could scan over, we only need to consider a finite number of them. This is because the function ϕ only depends on which points are contained in the shape, so the number of shapes considered is bounded as a function of the number of input points $n = |X|$. Although there is an exponential number of subsets 2^n, restricting to geometric shapes reduces this to a polynomial number: at most n^3 for disks and at most n^4 for rectangles. Brute-force

[2] Code and rendering scripts: `https://github.com/foadnamjoo/sampling-region-scan`

enumeration of these candidates, spending $O(n)$ to score each one, therefore takes $O(n^4)$ and $O(n^5)$ time for disks and rectangles, respectively; sophisticated algorithms reduce this to near-quadratic $O(n^2 \log n)$ for rectangle discrepancy (Agarwal et al., 2006b), which the `pyScan` implementation we use in Section 4 exploits (see Section 5 for measured runtimes).

Region-Based Spatial Scan Statistics In many real-world datasets, data are aggregated over spatial regions (e.g., counties), rather than available at individual locations. Let Z be a collection of regions, with each region $z \in Z$ associated with the baseline and measured values $b(z)$ and $m(z)$, respectively. For any subset $Y \subset Z$, we define $m(Y) = \frac{1}{M} \sum_{z \in Y} m(z)$ and $b(Y) = \frac{1}{B} \sum_{z \in Y} b(z)$, where $M = \sum_{z \in Z} m(z)$ and $B = \sum_{z \in Z} b(z)$; the same cost function $\phi(Y)$ then applies unchanged.

The key challenge is to define the candidate subsets Y (regions of interest). While a potential subset could be any connected set of regions, as in FlexScan (Tango and Takahashi, 2005; Takahashi and Tango, 2023), this is not tied to geometric shapes, and can potentially over-fit to strangely shaped regions. Also, it can be significantly slower than other scanning algorithms (Grubesic et al., 2014); also see Table 1 in Section 5.

A more common strategy is to use geometric shapes $\mathcal{S}$ (e.g., disks, rectangles) as before, but apply them to regions. This presents two main obstacles: First, in the point-based setting, one can identify a canonical shape from a valid subset $Y \subset \mathbb{R}^2$ as the *smallest* shape $S_Y \in \mathcal{S}$ that contains Y (and nothing from $X \setminus Y$). Given a set of points Y and $\mathcal{S}$ as disks or rectangles, the smallest shape can be found easily with computational geometry. However, with region-based input, where each $z \in Z$ has a complex geometry (e.g., a polygon in a shapefile), the computation of the minimal enclosing shape is more difficult. Second, geometric shapes $S \in \mathcal{S}$ are likely to be unable to partition Y from $Z \setminus Y$, meaning that they may partially overlap with regions $z \in Z$. It makes it more difficult to cleanly partition Z into Y and $Z \setminus Y$.

It is not immediately clear how to handle these in defining $b(Y)$ and $m(Y)$, at least not efficiently. One approach is the area-based framework of Buchin et al. (2012). They model the map as a polygonal subdivision with per-region case and population counts, and weight a regions contribution towards ϕ proportional to its overlap. Their method, however, only considers a fixed-scale polygonal window. They discretize the continuous placement problem by constructing the arrangement of combinatorially distinct placements of the polygonal shape with respect to the subdivision, and then optimize the likelihood within each cell. In experiments on real geography with synthetic cases, their area-based methods localized clusters more accurately than centroid-based baselines. They also introduced a non-homogeneous variant that showed lower detection power. In practice, circular windows are approximated by regular polygons. A limitation of their method is the fixed scale and aspect ratio (for rectangles) of anomalous regions they consider.

However, the most widely used heuristic to resolve these difficulties is to simply represent each region $z \in Z$ at a single point x_z at its centroid, assigning $b(x_z) = b(z)$ and $m(x_z) = m(z)$. This allows for direct application of point-based scan algorithms. However, this approximation introduces geometric inaccuracies: a shape S containing x_z may only partially intersect z, leading to a possible misalignment between the shape and the underlying region. This approach *ignores* this potential error.

2.1 Software for Spatial Scan Statistics

Several software tools implement the spatial scan statistic, each with trade-offs on speed, flexibility, and accuracy in detecting regions.

Probably the oldest and most widely used implementation is SaTScan (Kulldorff, 2022), which supports both point-based and region-based data. For regions, it reduces the regions to single points—typically centroids—and scans over circular or elliptical windows to maximize $\Phi_{\mathcal{S}}$. It offers a variety of statistical models, as well as searching for space-time, purely temporal, and seasonally-defined anomalies.

The pyScan (Matheny, 2024) offers a modern Python interface with significantly improved scalability. It relies on a well-engineered C++ backend with algorithmic enhancements for computing spatial scan statistics efficiently. Additionally, pyScan allows for guaranteed approximations of the optimal score, trading small controlled loss in accuracy for substantial gains in speed. In this work, we enable a mild approximation in pyScan; see Section 4.

Another option is FlexScan (Takahashi and Tango, 2023), which operates directly on a region-based input. It performs an exhaustive search over combinations of spatially connected regions to identify clusters. FlexScan allows for more flexibility in cluster shape, but several previous studies have highlighted critical limitations. Tango and Takahashi (2005) noted that FlexScan has high computational overhead, especially when applied to datasets with a large number of spatial regions; an efficient algorithm is needed. Moreover, it is limited to clusters of at most 15 regions; if the anomalous area is larger than that, it reports multiple disjoint regions without indicating if they are contiguous or not. In the large cluster case, it does not report a single region or single significance score. We also found that FlexScan requires more manual pre- and post-processing; for instance, if the dataset contained unconnected regions, it failed to report any clusters unless these were first removed.

We also obtained a Java implementation (JDK 17) of the area-based method of Buchin et al. (2012) by contacting the authors. The method scans a window on a *fixed* scale, as required by the code. Because the spatial extent of an anomaly is a key aspect of what one is trying to discover, this is a strong assumption. The paper recommends trying several scales (using 9 multiples $\{0.5, 0.7, 0.85, 0.95, 1.0, 1.05, 1.15, 1.3, 1.5\}$ of the guess – or given the true planted size) retaining the highest scoring anomaly across all scales, and we follow this in our experiments. The true size would not be known by a practitioner. By contrast, the scan methods of SaTScan and pyScan that our method extends are able to consider all scales from some shape family intrinsically.

3 Methods

To address the challenge of converting region-aggregated spatial data to be compatible with point-based scan statistics, we propose a principled alternative to the common centroid approximation.

Baseline: Centroid Method In the centroid-based approach, each region z is represented by a single point x_z located at its geometric centroid. The entire measured and baseline values of the region are then assigned to that point, i.e. $m(x_z) = m(z)$ and $b(x_z) = b(z)$. Although computationally efficient, this reduction discards the internal spatial structure of the region, potentially diminishing statistical power and introducing bias.

Our Proposed Sampling-Based Method Instead of using one centroid point per region, we propose replacing each region z with a set of k representative points: $x_{z,1}, \ldots, x_{z,k}$. These points are sampled uniformly at random from within the geometry of z. The region's values are then evenly distributed across these points, so that

$$m(x_{z,j}) = \frac{m(z)}{k}, \quad b(x_{z,j}) = \frac{b(z)}{k}, \quad \text{for all } j = 1, \ldots, k.$$

This strategy preserves both the spatial extent and the internal structure of the original region while remaining compatible with existing point-based scan algorithms.

While we explore other variants, we find that random selection of these k points in each region is effective. We explore the values of k from 1 (a single uniformly sampled point, i.e. Random Point; also the deterministic Centroid) to 50. Section 5.2 studies this choice empirically, finding diminishing returns beyond $k \approx 20$. Larger values of k tend to produce higher statistical power, as they better represent the geometry of each region and, in the synthetic experiments, provide more observations of the planted signal. We find that while the results improve with larger k, the full $k = 50$ points are not always needed. The optimal choice may depend on the complexity of the dataset and computational constraints. Figure 1 shows an example of this sampling-based conversion applied to counties in the state of Arkansas. The left panel uses $k = 10$ points per region, while the right panel uses $k = 50$.

3.1 Recovering Planted Anomalies

To evaluate the statistical power of the point-based methods, we consider their ability to recover anomalous regions that we synthetically plant in the data. We generate a planted shape $S \in \mathcal{S}$ and construct each point-based input: Centroid places one point at each region's geometric centroid, and Geom-k samples locations uniformly within each region. Conditional on these locations, we

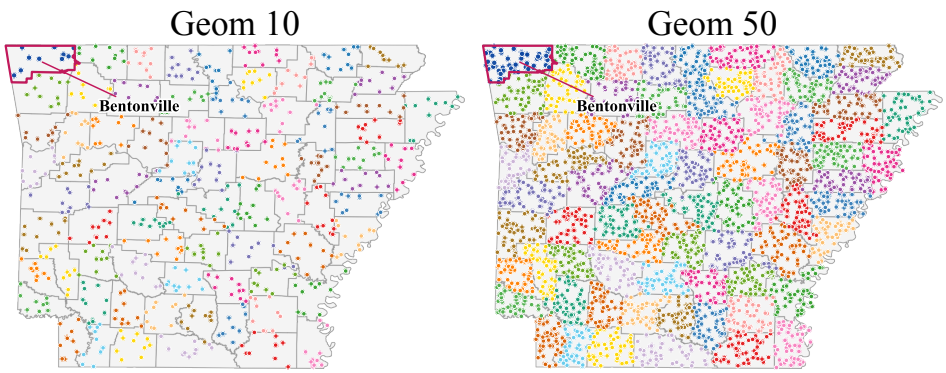

Figure 1: Region-to-point sampling on Arkansas counties: each county is replaced by k points drawn uniformly at random from its polygon. For example Benton County (outlined), contains exactly 10 points in Geom 10 (left) and exactly 50 points in Geom 50 (right).

independently include each point in the measured set with probability p if inside S and probability q if outside it. We then pass the measured and baseline point sets to a scanning algorithm such as pyScan, which maximizes the Poisson-derived Kulldorff score, and assess recovery using the discovered region $\hat{S}$. Our experiments primarily use planted rectangles and rectangular scan windows.

We evaluated how well algorithms work by how closely the identified anomalous region $\hat{S}$ (the *Discovered Rectangle*) matches the planted one S (the *Target Rectangle*). We do not expect these to match exactly due to spatial rounding errors and since each point is independently included in the measured set probabilistically. As rates p and q become similar, the most anomalous rectangle from the generated data becomes less distinct from the generating one.

To quantify the similarity between S and $\hat{S}$, we use a modified Jaccard distance defined on a large set of fixed points A. We construct the fixed evaluation set A by independently sampling 500 points uniformly within each input region, so $|A| = 500n$, where n is the number of regions. Using a separate fixed seed, we build A once per dataset and reuse it across all methods, values of k, rate contrasts, trials, and experiment seeds. This dense sampling ensures robust, reproducible overlap comparisons.

We define *Point Jaccard distance* as the following:

$$d_{\text{JAC},A}(S, \hat{S}) = 1 - \frac{|A \cap (S \cap \hat{S})|}{|A \cap (S \cup \hat{S})|}.$$

This distance is one minus the fraction of evaluation points in $S \cup \hat{S}$ that also lie in $S \cap \hat{S}$; lower values indicate greater overlap.

Figure 2 illustrates an example for Arkansas counties using the Geom 5 method. Lower Jaccard distance values indicate better recovery of the planted anomaly. This provides visual intuition for the meaning of various Jaccard distances. In particular, the last panel shows where the target region is a disk, not a rectangle; it has a Jaccard distance of 0.161. This is especially informative since in

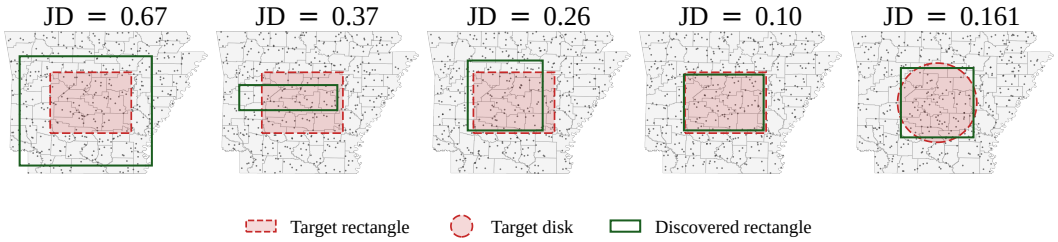

Figure 2: Point Jaccard distance between target (red) and discovered rectangles (green outline) for Arkansas counties. The last panel shows the best-fit discovered rectangle to a circular shape.

practice the true anomalies may not be rectangular, and this provides a sort of "shape floor" for how similar we should expect to recover a planted anomaly. That is, below about 0.2 Jaccard distance is probably sufficient. On the other hand a Jaccard distance above 0.35 is not a good fit.

4 Experimental Evaluation

Our main experiment evaluates how effectively each scanning method recovers a planted anomalous region under varying conditions. We vary the algorithm's parameters and the experimental setup to see how algorithms perform in different case studies. To capture a diverse range of spatial structures and region complexity, we conducted experiments on six different geospatial datasets. Arkansas, New York City, Utah, California, Georgia, and the entire United States. These cases represent a spectrum from local compact, densely populated regions (e.g., NYC) to large, sparsely populated ones (e.g., Utah).

Unless otherwise noted, each experimental result was produced by taking the average Jaccard distance of the 20 trials. Both the experimental setup and the newly proposed algorithms are random, so it is helpful to understand the distribution of the results. We also plot shaded regions that show the results' average values plus or minus one standard deviation. Methods with narrower bands are more consistent, whereas wider bands indicate a higher sensitivity to randomness in the setup.

Measuring Statistical Power While there are various ways to formulate it, *statistical power* for a method informally captures how reliably it can solve a task; in our case the probability of detecting a planted anomaly at a certain difficulty level. In the classical sense, power is the probability of rejecting the null hypothesis of no cluster when an anomaly is truly present, which presumes a significance test at a fixed level (Glaz and Koutras, 2024; Tango and Takahashi, 2005). We instead report how closely each method *localizes* a planted region, via Point Jaccard distance averaged over trials, and treat a method as having higher power when it recovers the region at smaller pq differences. The two notions are related but not interchangeable: a method could reject the null and still mislocate the cluster.

We vary problem difficulty, and measure the statistical power, using two factors: the pq *difference* and the *size of the planted region*. The pq difference (our primary focus) captures the contrast between the measured rates inside (p) and outside (q) the planted region. In our experiments, we always set $q = 0.2$, so for each baseline value $b(z)$ outside a planted region, we expect to observe $\mathsf{E}[m(z)] = (0.2)b(z)$ as the corresponding measured value. We vary the rate within the region from $p = 0.2$ (so the pq difference is 0) and $p = 0.9$ (so the pq difference is 0.7). The more different these values, the more apparent the anomaly should be, and the easier it should be to detect reliably. When $p = 0.9$, the difference ($p - q = 0.7$) should present an obvious anomaly; when $p = 0.2$, the planted region should be indistinguishable from the background.

A method with large statistical power can with high probability recover the planted region even at lower pq differences. Thus, we interpret the lower thresholds of p (with fixed q) at which recovery is still reliably successful as clear indicators of higher power.

Methods Considered We evaluated five point-based conversion strategies using the `pyScan` (with adaptive grid at size 100×100) and restrict our scan shapes to rectangles. We chose this shape because it provides the best combination of representational complexity and computational efficiency. We consider a region-based input and convert the problem to point-based input in 5 different ways. Centroid is the main baseline approach Kulldorff (2022), where each region is represented by a single point on its geometric centroid. Random Point replaces each region with one randomly sampled from within the region's shape. This is a variation of our proposed method but uses only a single point. This variant shares the same computational cost as the centroid method, but introduces randomness. Later we more carefully compare to FlexScan Tango and Takahashi (2005, 2012) and Buchin *et al.* Buchin et al. (2012).

Geom 5, Geom 10, and Geom 50 represent our proposed multi-point sampling strategy. They replace each region with 5, 10, or 50 randomly sampled points, respectively, distributing the region's baseline and measured values evenly across those points.

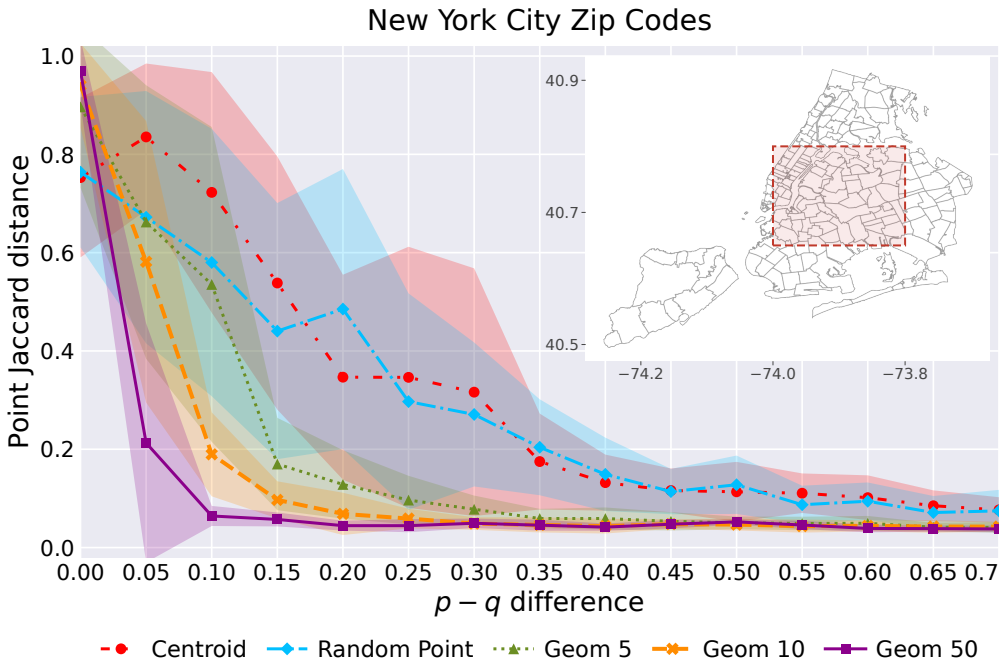

Figure 3: New York City with 263 polygons representing 248 unique ZIP codes. The inset (upper right) shows the planted target rectangle (red dashed). The Point Jaccard distance versus pq difference, averaged over 20 trials with ± 1 standard deviation bands.

4.1 Zip Codes in New York City

We start our evaluation by studying the zip code boundaries of New York City (263 polygons representing 248 unique ZIPs; ten ZIPs, e.g., 10004, are split into multiple polygons by water, and we treat each polygon as an independent region in the scan). These shape files vary significantly in size and geometry – although they are designed to represent zip codes with approximately equal populations. We assign the same baseline value to each zipcode region. We synthetically plant a rectangular region in a central portion of the map, covering approximately one-third of the total area. This planted region is shown inset in Figure 3. Using this shape, we fix $q = 0.2$, and vary the rate p inside the shape.

Figure 3 shows the Point Jaccard distance as a function of the pq difference on the x-axis for each method. Both the Centroid and Random Point approaches exhibit similar behavior: they are relatively noisy and consistently identify the planted region only when the pq difference is approximately 0.35 or 0.4. In contrast, as we increase the number of random points from 1 to 5, 10, or 50 per region, the results become more stable and the planted region is detected with much smaller pq differences.

Specifically, with 5 random points, the planted region is reliably recovered at pq differences of 0.2 or greater; with 10 points, detection occurs at pq differences as low as 0.1. With 50 points per region, recovery is close to the 0.2 threshold at $p - q = 0.05$, where Point Jaccard distance is 0.213, and is strong by $p - q = 0.10$, where the distance falls to 0.064.

Note that even when the pq difference is very large (e.g., at least 0.5), the Geom methods converge to a Point Jaccard distance of around 0.05, and Centroid and Random Point to around 0.1. Although we could potentially devise a way to measure the accuracy of the region recovery so that it converges to 0 error, this nonzero plateau is consistent with randomness in the synthetic data-generation process.

4.2 Counties of US States

Utah. Next, we consider the 29 counties in the state of Utah; see the inset panel of Figure 4. This example has few counties which are often irregularly shaped, which introduces additional geometric challenges for region-based analysis.

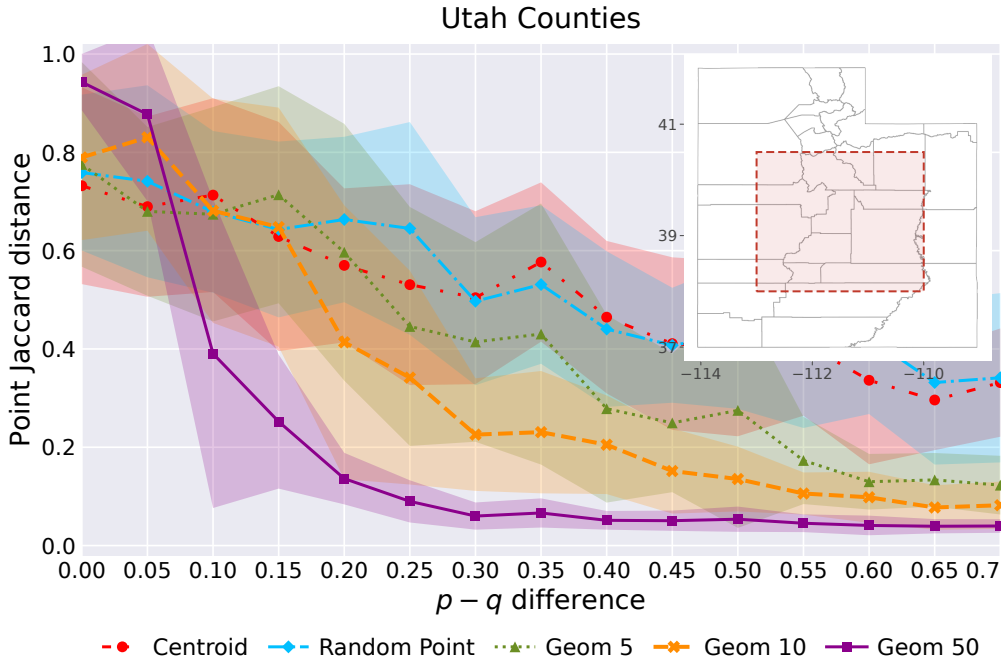

Figure 4: Utah with 29 counties. The inset (upper right) shows the state and the planted target rectangle (red dashed). The Point Jaccard distance versus pq difference.

Figure 4 shows, due to the number of regions and their complex shapes, substantial variability in the Point Jaccard distance for both the Centroid and Random Point approaches, as well as for Geom 5. These methods struggle to consistently detect the planted region, particularly at low pq differences.

The Geom 10 and Geom 50 approaches show more reliable behavior. Although some instability persists at small pq differences, their performance stabilizes as the pq difference increases.

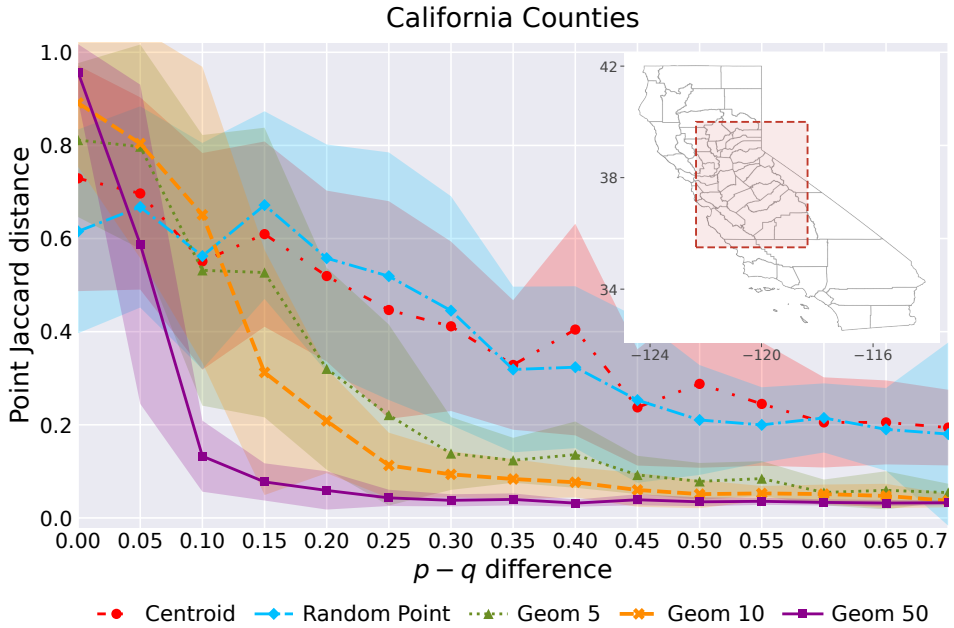

Figure 5: California, 69 polygons over 58 counties. The inset (upper right) shows the state and the planted target rectangle (red dashed). The main plot shows Point Jaccard distance.

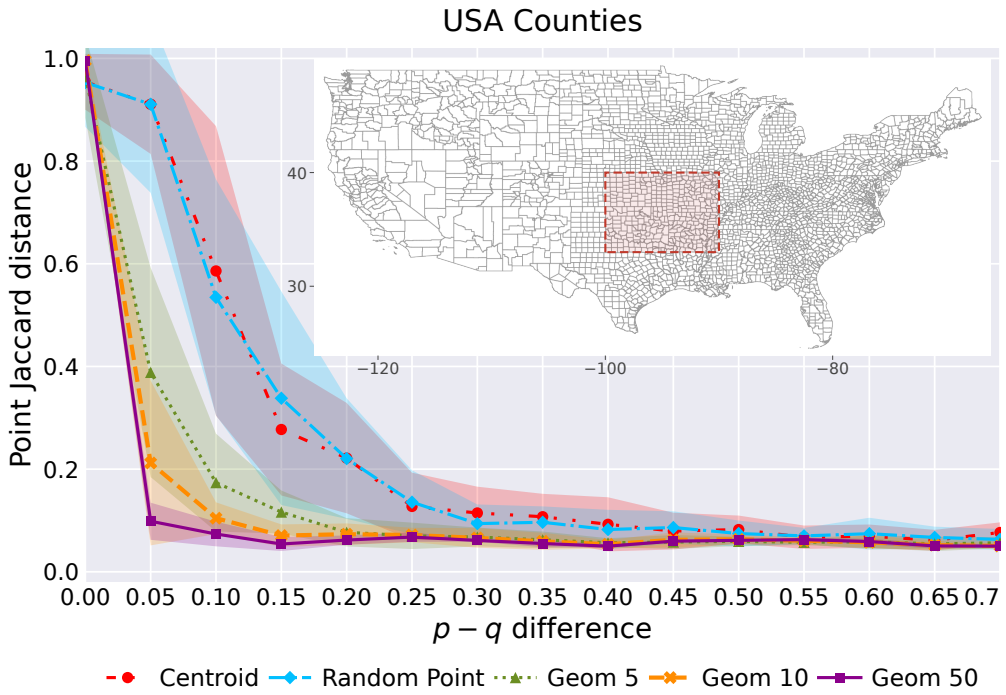

Figure 6: Point Jaccard distance for 3,108 counties in the contiguous United States. The inset shows the country and the planted target rectangle.

California. The counties in the state of California present a distinct challenge for spatial analysis. Although there are 58 counties[3], the state's shape is oblong and the counties are of vastly different sizes, introducing spatial heterogeneity and complicating detection. The scenario we consider, shown in Figure 5, has the target rectangle that separates several smaller counties in the Bay Area and even includes areas outside the state boundaries.

The results highlight the effectiveness of our proposed method. Geom 50 consistently achieves lower Point Jaccard distances across a range of pq differences, indicating high statistical power. While the other methods, including the centroid approach, can achieve a small Point Jaccard distance, they have much more variation in what they find, demonstrated by much larger standard deviation.

Contiguous USA. We evaluate our methods on the 3,108 counties in the contiguous United States; Figure 6 shows the results. The larger number of regions and the increased spatial coverage create a more stable statistical environment, allowing all methods to perform more reliably.

As shown in Figure 6, all methods benefit from the larger number of regions, but the Centroid method improves more slowly and remains above 0.1 at a pq difference of 0.3. The proposed methods exhibit more statistical power: Geom 50 reaches approximately 0.10 at a pq difference of 0.05, while Geom 10 is approximately 0.21 at 0.05 and approximately 0.10 at 0.1.

4.3 Effect of Target Rectangle Size

To further evaluate robustness, we conducted a different experiment to show how the methods are affected by the size of the planted region. We started by selecting a small rectangle and progressively increasing its size from 2% to 61% of the state of Georgia; see Figure 7. In this experiment, we fixed the pq difference at 0.4 to isolate the effect of the region size on the statistical power.

As the target rectangle becomes smaller, it contains fewer anomalous signals, making it more difficult to distinguish it from natural random variation elsewhere. The results in Figure 7 show that the

[3]California has 58 counties, however our shapefile splits three coastal counties by their offshore islands leading to 69 polygons, which we treat as independent regions.

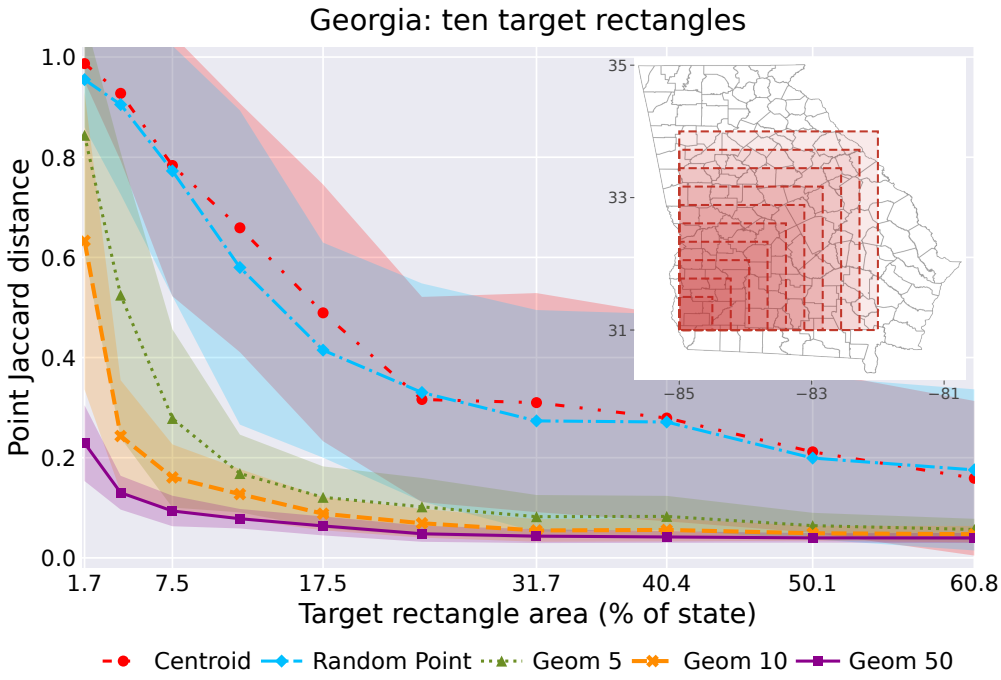

Figure 7: Point Jaccard distance as a function of the relative size of the planted target rectangle in Georgia, with $p - q$ fixed at 0.4 and averaged over 80 trials. The inset shows the ten nested target rectangles overlaid on Georgia counties.

Centroid and Random Point methods struggle across all region sizes, with performance degrading substantially for regions smaller than 12% of the state.

In contrast, our proposed methods recover substantially smaller targets. At the 0.2 recovery threshold, Geom 5, Geom 10, and Geom 50 recover targets covering as little as approximately 12.0%, 7.5%, and 4.1% of the state, respectively. By comparison, Random Point first reaches the threshold at approximately 50.1%, while Centroid reaches it only at the largest tested target size, 60.8%. This highlights that adding even a few random points (instead of 1) in each region can greatly enhance the statistical power of the methods to detect significantly small spatial anomalies.

4.4 Comparison with FlexScan and Buchin et al.

We also evaluate our proposed method in counties in the state of Arkansas and compare its performance with the FlexScan[4] technique (a brown curve), and the area-based method of Buchin et al. (Buchin et al., 2012) (a dark blue curve). FlexScan identifies a connected region of counties with elevated rates. The Arkansas dataset includes a moderate number of counties (75), each with fairly uniform shapes, making it a favorable setting for FlexScan. As in previous experiments, we plant a rectangular anomaly, this time covering approximately 30% of the state; see Figure 8.

The plot in Figure 8 shows how the methods perform as the pq difference increases. We observe that the performance of the Centroid, Random Point, and Geom 5, 10, 50 methods follows trends similar to those of previous experiments.

FlexScan While FlexScan performs reasonably, its Point Jaccard distance plateaus around 0.35–0.4 (above the 0.2 practical recovery threshold), even as the pq difference increases. This appears to be an artificial implementation limitation, as the code limits the search for the potentially anomalous cluster to at most 15 regions.

To ensure a fair comparison, we repeated the experiment on the Arkansas dataset using a smaller target rectangle covering only 10% of the area, results shown in Figure 9. This setup is more challenging

[4]FleXScan v3.1.2 software manual instructions (Takahashi and Tango, 2023)

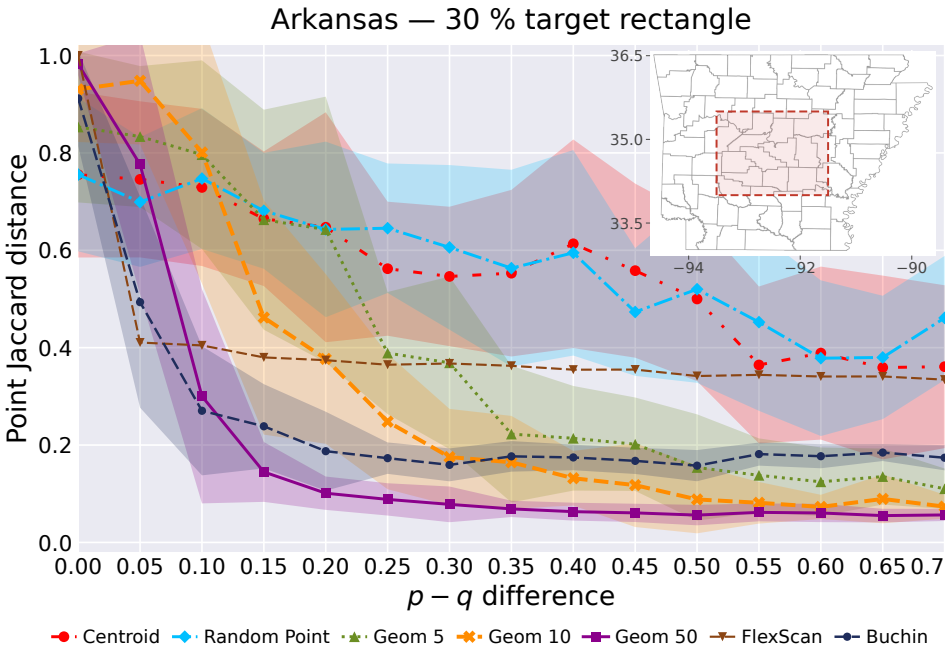

Figure 8: Point Jaccard distance between the planted target and the discovered region for Arkansas, with the target rectangle covering about 30% of the state.

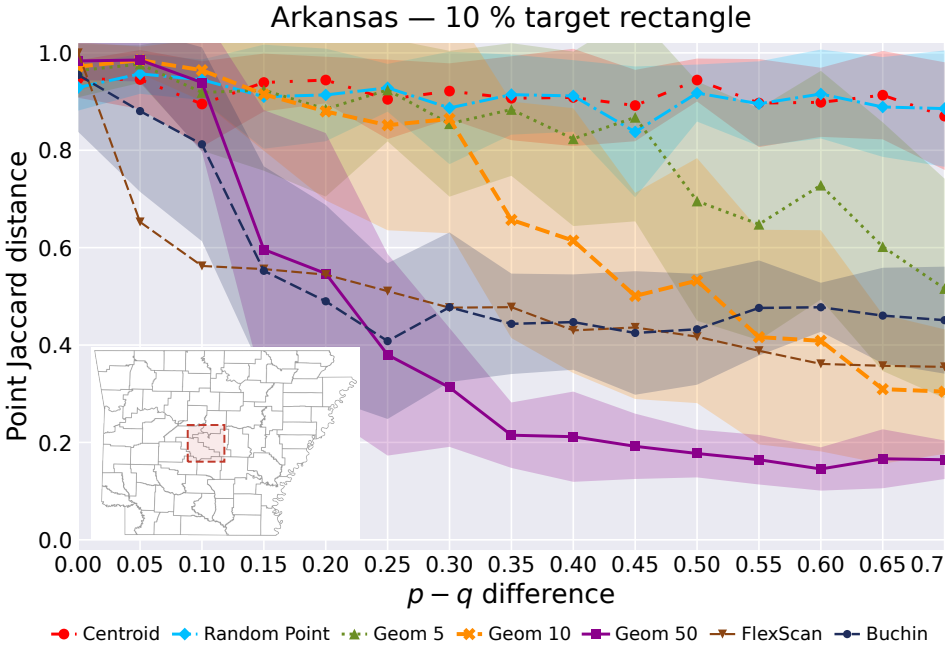

Figure 9: Point Jaccard distance between the planted and discovered regions for Arkansas, with the target rectangle covering about 10% of the state.

for both the Centroid and our proposed methods. Nevertheless, Geom 50 remains the best performer, achieving a Point Jaccard distance of approximately 0.2 (or less) for $pq \geq 0.35$. FlexScan, on the other hand, fails to achieve a Point Jaccard distance below 0.35, even as pq increases. Notably, for very small pq differences (≤ 0.1), FlexScan outperforms our methods. Still, this advantage is limited, as the Point Jaccard distance is still quite high (Jaccard distance ≈ 0.6), indicating that the overlap between the target and the discovered regions has some overlap on average, but not much.

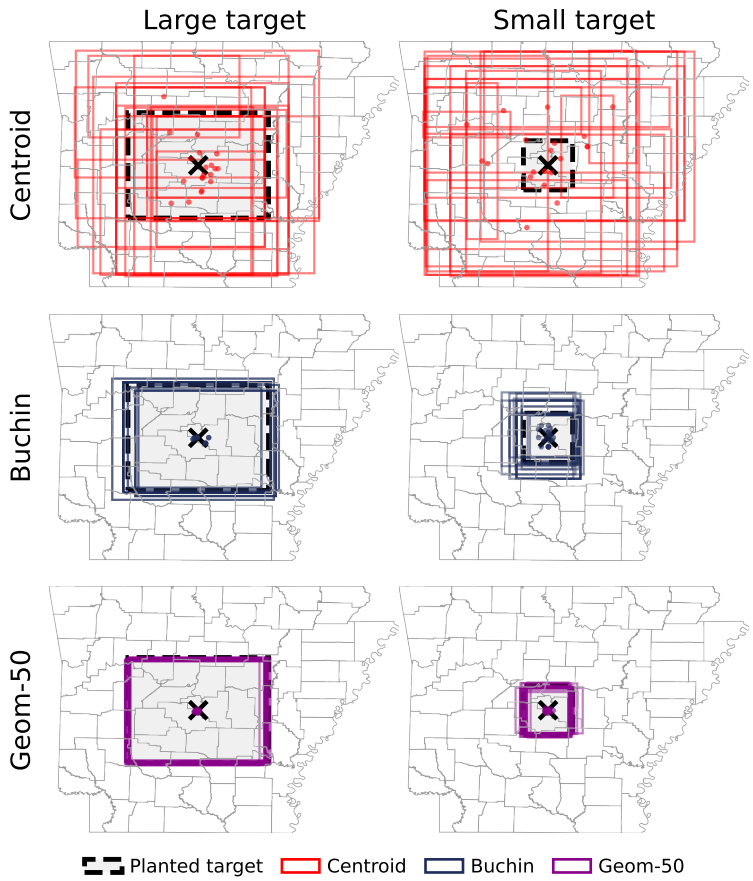

Figure 10: Discovered rectangular windows on Arkansas for planted targets (black dashed) at $p - q = 0.5$, 20 trials each. **Rows: Centroid** (red), **Buchin** (navy), and Geom 50 (magenta).

Buchin. On the larger target (Figure 8) the area-based method of Buchin et al. (Buchin et al., 2012) tracks Geom 50 closely but with on average a point Jaccard 0.1 larger, and selecting nearly the correct scale (on average $1.07\times$ the planted area; offset ~ 6.0 km) even though it was given the planted size. On the smaller target (Figure 9) the two methods have a larger gap: Buchin systematically oversizes to on average $1.61\times$ planted (offset ~ 11.4 km) and plateaus near 0.45, while Geom 50, whose continuous search adapts the scale, stays at $0.95\times$ planted (offset ~ 3.1 km) and reaches 0.16 for Point Jaccard. Figure 10 shows these discovered rectangles geographically. Unlike Buchin *et al.*'s area-based method, our approach requires no user-supplied window size and is roughly $90\times$ faster on Arkansas (Section 5, Table 1). A comparison on *disk-shaped* planted targets, where pyScan can match the target exactly, is reported in Appendix B.

4.5 Ablation on Sampling and Similarities

Finally, we consider two design choices: (1) the similarity metric used to compare the target and discovered regions, and (2) the sampling points strategy within each region. Our findings indicate that, while these choices introduce slight differences in performance, the selected default settings yield consistently strong results.

Our main evaluation metric, the *Point Jaccard distance*, is based on comparing point sets inside each region. Specifically, we sample $500n$ points from the n regions and compute the Jaccard distance between the sets associated with the target and discovered regions.

As an alternative, we could consider the *Area Jaccard distance*, which compares regions directly via geometric overlap. This metric is defined as one minus the ratio of the intersection area to the union area between the target and discovered regions. It provides a shape-based similarity score.

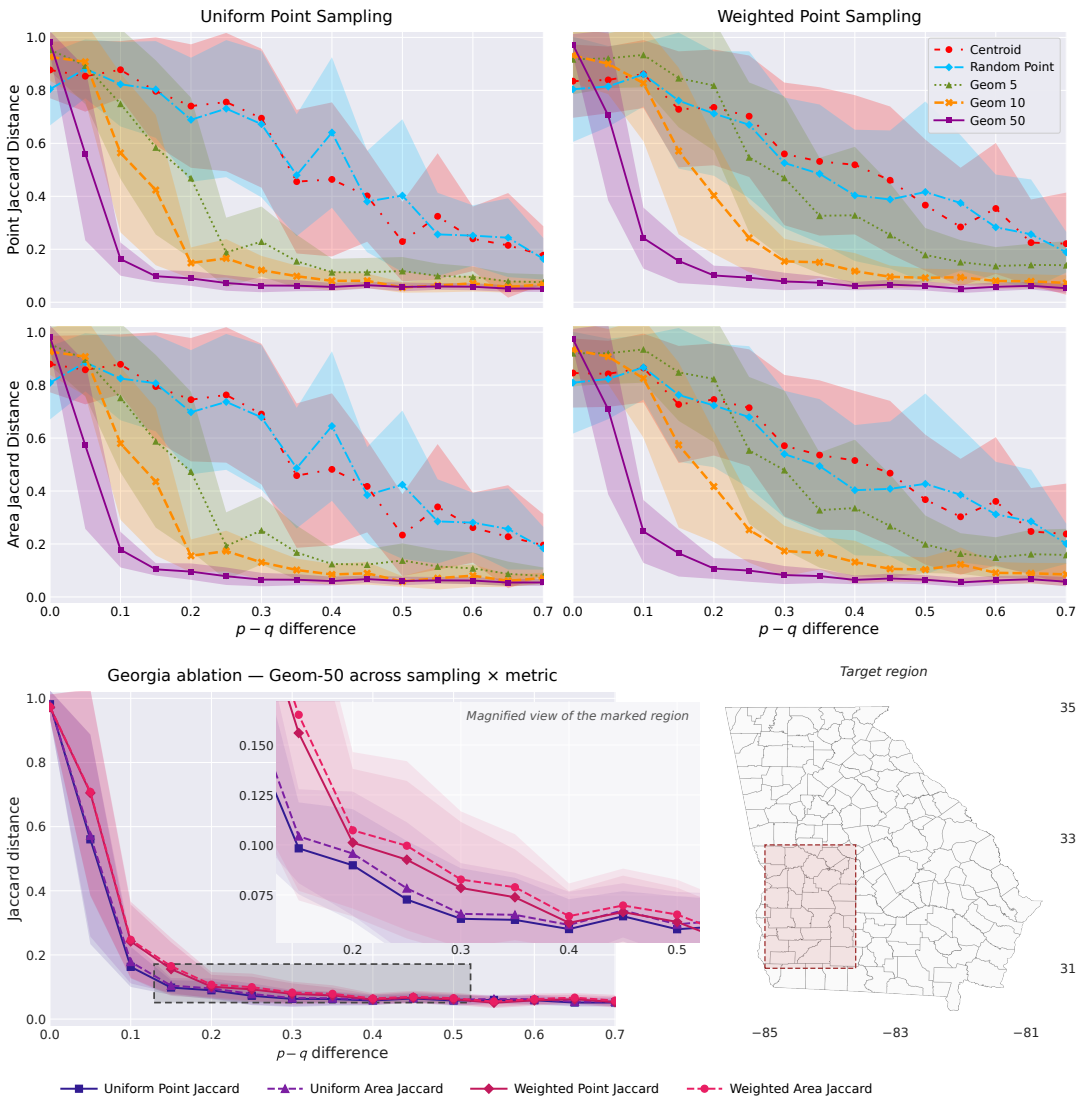

Figure 11: Georgia counties with a planted region covering approximately 30% of the state. **Top:** the 2×2 grid shows Point Jaccard distance (top row) and Area Jaccard distance (bottom row) under uniform point sampling (left column) and capped weighted point sampling (right column) across the five methods. **Bottom:** focused comparison of Geom 50 across the four (sampling, metric) settings, with the magnified panel zooming into $p - q \in [0.15, 0.50]$.

We also compare two strategies for sampling points within each region: *Uniform Sampling:* Each region receives an equal number of sampled points (e.g., 1, 5, 10, or 50). *Weighted Sampling:* Each region receives a method-specific minimum number of points, with more points assigned to more populous regions, up to an upper cap. This aims to reflect a higher signal strength in denser regions.

While weighted sampling might seem more natural, it can lead to poor performance in sparse regions. For example, in the Georgia dataset, densely populated areas such as Atlanta are capped, while smaller or rural counties receive few points—diminishing the ability to detect target rectangles that span low-density areas.

Figure 11 shows all four combinations in Georgia counties, using Geom 50 on a single plot using a target rectangle covering about 30% of the area. It again shows the two uniform sampling settings performing slightly better for small values of the pq difference, though the two arms do not use the same number of points.

USA	Geom 1	Geom 5	Geom 10	Geom 50	Buchin	FlexScan
Runtime (s)	0.29	0.29	0.29	0.33	114.0	1109
St.dev.	0.001	0.001	0.002	0.002	4.4	–
Arkansas						
Runtime (s)	0.21	0.26	0.27	0.28	25.3	8.00
Std	0.015	0.005	0.002	0.002	0.12	–

Table 1: Runtime in seconds for one region discovery, scan step only. Top: 3,711 U.S. county-by-congressional-district regions; bottom: 75 Arkansas counties. Geom-k and Buchin report means over five runs.

Among the top 4 plots, the first row uses the Point Jaccard distance and the second row uses the Area Jaccard distance. The first column uses uniform point sampling, and the second uses capped weighted point sampling. While there are some differences among these four plots, all give the same general effect. Perhaps the only significant signal is that under uniform point sampling, Geom 5 and Geom 10 more consistently recover the target region for pq difference in the range of 0.15 to 0.25, when compared to using weighted sampling.

5 Runtime and Convergence

Converting a problem with n input points to $50n$ (as we did in Geom 50) points could lead to a significant computational slowdown, changing problems from tractable to intractable. Although naive algorithms for computing spatial scan statistics over rectangles on n points scale with $O(n^4)$ or $O(n^5)$, there are improvements that reduce this to near quadratic in n; Agarwal *et al.* (Agarwal et al., 2006b) show how to compute this in $O(n^2 \log n)$ time. An efficient version of this algorithm is implemented in pyScan, which we employ in this paper. Even with this optimized implementation, a $50\times$ increase in data could imply a theoretical $2{,}500\times$ slowdown. Moreover, we leverage mild approximation in pyScan setting its `max_subgrid` to an adaptive resolution grid at 100×100. Its running time now partially depends on this grid size $g = 100$ as $O(g^2 \log g + n)$ Matheny and Phillips (2018). Increasing the points per region therefore has a more limited effect on runtime. Concretely, going from Arkansas ($n = 75$) to the USA ($n = 3{,}711$ county-by-congressional-district regions) is a $50\times$ increase in regions, yet the scan time grows only about $1.2\times$ (Table 1).

All runtimes were measured as wall-clock time on an Apple M2 Pro (8 performance + 4 efficiency cores), 16 GB RAM, macOS, Python 3.10. They consider a single region discovery at $p = 0.6$, $q = 0.2$, measuring only the scan step (case generation and data loading excluded). Buchin *et al.*, as default, considers 9 sizes chosen around the planted size. FlexScan's number additionally includes a Monte Carlo significance test that the other methods do not perform; but we were unable to decouple in running the code.

Table 1 shows that pyScan's scan time is dominated by the fixed-resolution grid scan, so Geom k for $k \in \{1, 5, 10, 50\}$ all complete one discovery in about 0.3 seconds on the USA dataset ($n = 3{,}711$ regions). While we demonstrate this with pyScan, other efficient scanning approaches, like those by Neill and Moore (Neill and Moore, 2004), would also keep this manageable. Buchin's rectangle scanner takes 114 seconds per discovery on the USA dataset — about $350\times$ slower than Geom 50 — because it sweeps a nine-point size grid and performs a translation search at each size. FlexScan takes $1{,}109$ seconds on the USA dataset (about 18 minutes), although that number includes a Monte Carlo significance test the other methods do not perform. As discussed in Section 2.1, FlexScan also presents interpretability challenges on large datasets, returning multiple clusters that are not straightforward to reconcile.

To complement our large-scale analysis, we also evaluated runtimes on the smaller Arkansas dataset, using a planted region that covers approximately 30% of the state; bottom part of Table 1. This setup aligns with prior evaluation studies (Tango and Takahashi, 2005, 2012) using FlexScan. Geom 50 completes a single discovery in approximately 0.28 seconds; Buchin's rectangle scanner takes 25 seconds (about $90\times$ slower), and FlexScan takes 8 seconds, including its Monte Carlo significance test. These results demonstrate that our method's runtime advantage holds across both small and large region counts.

Finally, pyScan also offers more aggressive approximation methods that compute the approximately optimal spatial scan statistic. These employ algorithms that involve subsampling data in addition to adaptive gridding, and other methods to scan them efficiently. In particular, these methods use coresets (Phillips, 2017), which first reduce the dataset problem to a size that depends only on the desired accuracy guarantee. As a result, the initial size of the dataset no longer becomes a runtime constraint (other than the time to store and sample from it), only the desired degree of accuracy. In this context, the proposed approach will not meaningfully impact the runtime.

5.1 Convergence

Our experiments demonstrate that sampling multiple points per region improves the statistical power, enabling more accurate recovery of true anomalous regions. However, we have not yet provided a theoretical explanation of why these methods work. In this section, we offer a stylized analysis to provide such insight. As with any stylized analysis, it will be an imperfect model of real-world settings, but we hope it provides some insight nonetheless. There are two key components to understanding how accuracy improves with the number k of sample points per region in Geom k. First, how accurately we can approximate the function ϕ, which the scan statistics algorithm seeks to optimize. Second, how this approximation translates to the quality of the recovered region, measured using Jaccard distance d_{JAC}. We address the second issue first. The scan statistic problem is a non-convex optimization problem with the potential for many local minima (e.g., see conditional hardness results (Matheny and Phillips, 2018; Backurs et al., 2016)), so unless we make some structural assumptions about the problem, even small approximation errors in ϕ could lead to a significantly different optimal shape, resulting in a large change in Jaccard distance. To address this, we analyze what we call *(ε, γ)-stable shapes*. An optimal shape $S^* \in \mathcal{S}$ is (ε, γ)-stable if for any $S' \in \mathcal{S}$ such that $d_{\text{JAC}}(S^*, S') > \gamma$ it holds that $\phi(S^*) - \phi(S') > \varepsilon$. This stability assumption implies that any shape with a score within ε of the optimum must be within γ in Jaccard distance.

While this assumption cannot be guaranteed for all real-world scenarios, in our simulated setting – with its controlled random generation process on a known and fixed planted shape S – we can satisfy that the optimal region S^* is (ε, γ)-stable for some choice of ε and γ; at least with a large enough sample size and *pq* difference. Thus, for our theoretical analysis, we assume to be seeking an (ε, γ)-stable shape; we will show that we can achieve ε-approximation in the cost ϕ, and this will imply we also obtain γ-approximation in the Jaccard distance.

To ensure that the recovered region $(\hat{S})$ is close to the true optimal region (S^*), we require that the difference in their scan statistic score is at most ε, that is

$$|\phi(\hat{S}) - \phi(S^*)| \leq \varepsilon.$$

In other words, we want the discovered region $\hat{S}$ to approximate the optimal region S^*, within a small margin of error in terms of the score function ϕ. To apply this in our setting, we need one additional concept, since the regions are imbalanced in size. Hence, we may need more points from the larger regions (e.g., more samples from Texas than Delaware in a US state dataset). To allow for analysis of our proposed algorithm, we consider an input of a set of regions $\mathcal{R}$ to be *κ-balanced* if $\max_{R \in \mathcal{R}} m(R) \leq \kappa \min_{R \in \mathcal{R}} m(R)$ and $\max_{R \in \mathcal{R}} b(R) \leq \kappa \min_{R \in \mathcal{R}} b(R)$. We can now state the following.

Theorem 1. *Consider an (ε, γ)-stable anomalous shape S^* in a κ-balanced set of n regions. If we use Geom k to sample $k = O(\kappa/(n\varepsilon^2))$ uniformly distributed points from each region and then run a scan statistics algorithm to obtain a shape $\hat{S}$, then with a constant probability we can guarantee $d_{\text{JAC}}(S^*, \hat{S}) \leq \gamma$.*

Proof. We rely on a probabilistic guarantee established in prior work (Matheny et al., 2016; Matheny and Phillips, 2018) that we can ensure an accurate estimate of ϕ under sampling. In particular, if we consider a random sample $X' \subset X$ of size $|X'| = O(1/\varepsilon^2)$, then with constant probability, the optimal scan statistic computed on X' will have a solution $\hat{S} = S'^*$ so that $|\phi(S'^*) - \phi(S^*)| \leq \varepsilon$, where S^* is the optimal solution of the full dataset X.

To apply these results in our settings, we treat the input X as a continuous distribution – in particular, a uniform measure over each input region. In our application, X represents either the measured value m or the baseline value b; where the density assigned to each region z is given by $m(z)$ or $b(z)$,

respectively. There is nothing in the above results (Matheny et al., 2016; Matheny and Phillips, 2018) that prevents X from being a continuous distribution, as long as we can draw random samples from it. Since we can draw uniformly from each shape file region, this modeling assumption is justified. The remaining challenge is that the required $O(1/\varepsilon^2)$ samples must be drawn from the full dataset, proportionally to the baseline b (and likewise m). This means that if there are n regions, we require $kn = O(1/\varepsilon^2)$ total samples, $k = O(1/(n\varepsilon^2))$ points – indicating as we subdivide into a larger number of regions n, the total number of samples stays fixed, and thus the number of samples needed per region decreases.

However, this $k = O(1/(n\varepsilon^2))$ bound assumes the regions are comparable in size. However, since we assume the regions are κ-balanced, then, if we sample $k = O(\kappa/(n\varepsilon^2))$ points per region, it will over-sample from some smaller regions, but guarantee to obtain the equivalent of the requisite (at most κ/ε^2) number of samples even in the largest regions.

Piecing together these components and transferring ε-error on ϕ to γ-Jaccard error via the (ε, γ)-stable assumption completes the proof. □

Perhaps surprisingly, this analysis indicates that as the number of regions n increases, then the number of samples per region k (the parameter in Geom k), *decreases*, proportional to $(\kappa/\varepsilon^2)/n$. To make sense of this, if the area of study (say all of the continental USA) is fixed, but the size of each region decreases (e.g., from states to counties to zip codes), then it is already gaining more resolution. Thus to cover the same area, there are fewer points per region k needed to allow the scanning to obtain the same refinement.

5.2 Choice of k

We further empirically explore the effect of the choice of k (uniform samples per region) and the recommendation of $k \in [20, 50]$ as effective and efficient. We sweep k from 2 to 100 on all six datasets in Figure 12. So a state with n regions contributes kn total points to the scan input. We fix the rate contrast at the pq difference $= 0.15$ and report the Point Jaccard distance averaged over 60 trials.

For most datasets Point Jaccard distance drops sharply until $k \approx 20$, after which returns diminish, although the large $n = 3{,}108$ USA dataset is already below the 0.2 recovery threshold by $k = 3$.

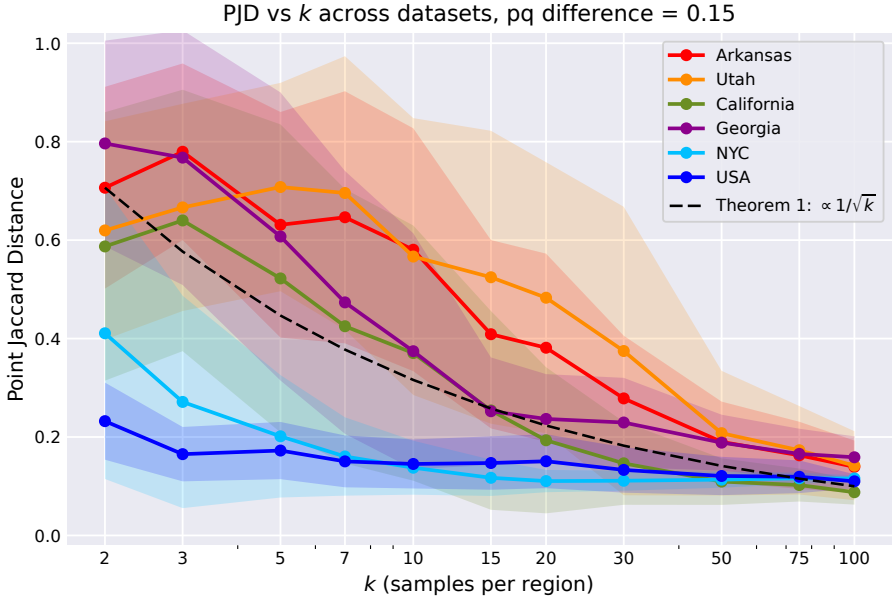

Figure 12: Point Jaccard distance as a function of k on Geom k, at fixed pq difference of 0.15, across six datasets. Curves show means over 60 trials; shaded bands show ± 1 std.dev.

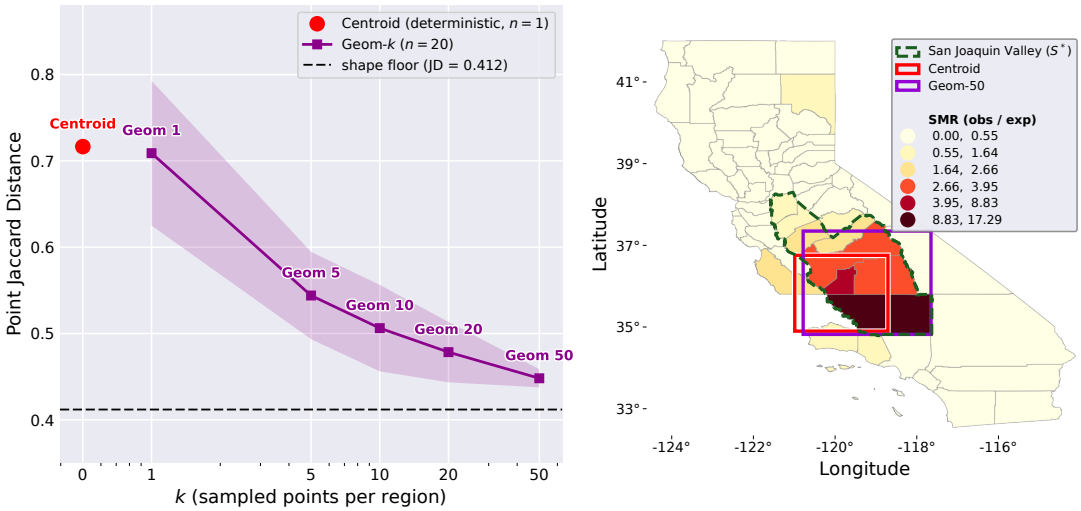

Figure 13: California Valley Fever: Point Jaccard distance to the San Joaquin Valley ground truth. The centroid (k=0, red) is shown against Geom-k for $k \in \{1, 5, 10, 20, 50\}$ (purple, mean ± 1 std band), shape floor (dashed). Right shows standardized morbidity ratio (SMR, observed/expected cases) by county, with the SJV ground truth $S^\star$ outlined (green dashed) and the regions found by Centroid and Geom-50.

Note that Theorem 1 gives that $k = (\kappa/n)(1/\varepsilon^2)$ samples are sufficient to achieve ε error (for a fixed dataset, with κ, n fixed). That means error (ε) drops proportional to $1/\sqrt{k}$. This rate is also plotted in Figure 12 as dashed black, pegged to the Arkansas dataset at $k = 2$. We observe that this general shape roughly applies to the various datasets; however the constants differ a bit as does n; with the large n datasets (USA and NYC) requiring fewer samples, as expected. Moreover, this does not converge to zero: by $k = 100$ the curves flatten between about 0.09 and 0.16. There is other variation that is harder to pinpoint such as that captured by κ.

6 Real Example: California Valley Fever

To validate the effect on real data, we run on county-level Valley Fever (coccidioidomycosis) incidence in California, whose elevated region has a known cause independent of the case counts. The fungal pathogen *Coccidioides* lives in arid soils, and its California endemic zone of San Joaquin Valley (SJV), centered on Kern County, and mapped from soil ecology Centers for Disease Control and Prevention (2024); California Department of Public Health (2024a). We set $m(z)$ to confirmed cases and $b(z)$ to population per county (CDPH, 2014–2018 California Department of Public Health (2024b)), take the ground truth $S^\star$ to be the eight SJV counties, run the Kulldorff scan over axis-aligned rectangles, and score each discovered region by its Point Jaccard distance to $S^\star$.

The centroid attains PJD $= 0.717$; Geom-k improves monotonically to 0.448 at $k = 50$, a gap of $+0.268$ (Figure 13). This lands only 0.036 above the shape floor of 0.412: the best overlap an axis-aligned rectangle can achieve, since the diagonally oriented valley cannot be wrapped tightly. In contrast, the centroid is stranded 0.305 above it. The discovered region sits inside the SJV with standardized morbidity ratio (observed/expected cases) ≈ 8 at every k, and the per-trial spread narrows as k grows. The valley's few very large counties (notably Kern) carry most of the case mass; collapsing each to its centroid destroys the information needed to *size* the region, while sampling restores it.

The improvement is robust to both natural design choices. Tightening $S^\star$ to the five hyperendemic core counties widens it to $+0.285$ (floor 0.294). Varying the window across a pre-surge period (2011–2013), the headline period (2014–2018), and the peak-outbreak year (2017) keeps it between $+0.243$ and $+0.268$, despite a $1.8\times$ swing in annual case load. The separation therefore reflects region geometry, not case volume.

7 Conclusion

We introduce a simple yet effective method for converting region-based inputs to point-based representation, enabling efficient algorithms for spatial scan statistics. Our approach, sampling multiple points uniformly within each region, significantly improves statistical power compared to standard techniques that rely solely on region centroids. Through planted-region experiments, we demonstrated that our method can recover anomalous regions under more subtle effect sizes while remaining computationally tractable.

Leveraging efficient implementation such as pyScan, we showed that even with increased point counts (e.g., Geom 50), runtime remains practical across both small and large datasets. This supports the scalability of our method without sacrificing precision or speed.

Given its ease of implementation, superior performance, and compatibility with existing algorithms, we recommend this sampling-based approach as the default way to convert region-based input to point-based representations for spatial scan statistics.

References

Ali Abolhassani and Marcos O Prates. 2021. An up-to-date review of scan statistics. *Statistic Surveys* 15 (2021), 111–153.

Deepak Agarwal, Andrew McGregor, Jeff M Phillips, Suresh Venkatasubramanian, and Zhengyuan Zhu. 2006a. Spatial scan statistics: approximations and performance study. In *Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining*. 24–33.

Deepak Agarwal, Jeff M. Phillips, and Suresh Venkatasubramanian. 2006b. The Hunting of the Bump: On Maximizing Statistical Discrepancy. *SODA* (2006), 1137–1146.

Arturs Backurs, Nishanth Dikkala, and Christos Tzamos. 2016. Tight Hardness Results for Maximum Weight Rectangles. In *43rd International Colloquium on Automata, Languages, and Programming (ICALP 2016)*, Vol. 55. Schloss Dagstuhl, Dagstuhl, Germany, 81.

Kevin Buchin, Maike Buchin, Marc van Kreveld, Maarten Löffler, Jun Luo, and Rodrigo I. Silveira. 2012. Processing aggregated data: the location of clusters in health data. *GeoInformatica* 16, 3 (2012), 497–521. `https://doi.org/10.1007/s10707-011-0143-6`

California Department of Public Health. 2024a. Coccidioidomycosis (Valley Fever). `https://www.cdph.ca.gov/Programs/CID/DCDC/Pages/Coccidioidomycosis.aspx`. Accessed 2026-06-05.

California Department of Public Health. 2024b. Infectious Diseases by Disease, County, Year, and Sex. California Health and Human Services Open Data Portal. `https://data.chhs.ca.gov/dataset/03e61434-7db8-4a53-a3e2-1d4d36d6848d`. Accessed 2026-06-05.

Centers for Disease Control and Prevention. 2024. Valley Fever (Coccidioidomycosis): Areas Where It Lives. `https://www.cdc.gov/valley-fever/`. Accessed 2026-06-05.

Michael R Desjardins, Alexander Hohl, and Eric M Delmelle. 2020. Rapid surveillance of COVID-19 in the United States using a prospective space-time scan statistic: Detecting and evaluating emerging clusters. *Applied geography* 118 (2020), 102202.

Joseph Glaz and Markos V Koutras. 2024. *Handbook of scan statistics*. Springer.

Tony H Grubesic, Ran Wei, and Alan T Murray. 2014. Spatial clustering overview and comparison: Accuracy, sensitivity, and computational expense. *Annals of the Association of American Geographers* 104, 6 (2014), 1134–1156.

Mingxuan Han, Michael Matheny, and Jeff M Phillips. 2019. The kernel spatial scan statistic. In *Proceedings of the 27th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*. ACM, New York, NY, USA, 349–358.

Martin Kulldorff. 1997. A spatial scan statistic. *Communications in Statistics-Theory and methods* 26, 6 (1997), 1481–1496.

Martin Kulldorff. 2022. *SatScan User Guide* (10.1 ed.). http://www.satscan.org/.

Martin Kulldorff, Lan Huang, Linda Pickle, and Luiz Duczmal. 2006. An elliptic spatial scan statistic. *Statistics in medicine* 25, 22 (2006), 3929–43.

Michael Matheny. 2024. *pyScan*. https://github.com/michaelmathen/pyscan.

Michael Matheny and Jeff M. Phillips. 2018. Computing Approximate Statistical Discrepancy. *ISAAC* 123 (2018), 7:1–7:14.

Michael Matheny, Raghvendra Singh, Liang Zhang, Kaiqiang Wang, and Jeff M. Phillips. 2016. Scalable Spatial Scan Statistics Through Sampling. In *SIGSPATIAL*. ACM, New York, NY, USA, 1–10.

Daniel B. Neill and Andrew W. Moore. 2004. Rapid Detection of Significant Spatial Clusters. In *KDD*. ACM, New York, NY, USA, 256–265.

Daniel B. Neill, Andrew W. Moore, and Gregory F. Cooper. 2006. A Bayesian Spatial Scan Statistic. In *NIPS*. MIT Press, Cambridge, MA, USA, 1003–1010.

Mallory Nobles, Ramona Lall, Robert W Mathes, and Daniel B Neill. 2022. Presyndromic surveillance for improved detection of emerging public health threats. *Science Advances* 8, 44 (2022), eabm4920.

Ganapati P Patil and Charles Taillie. 2004. Upper level set scan statistic for detecting arbitrarily shaped hotspots. *Environmental and Ecological statistics* 11 (2004), 183–197.

Jeff M Phillips. 2017. Coresets and sketches. In *Handbook of discrete and computational geometry*. Chapman and Hall/CRC, 1269–1288.

Shino Shiode. 2011. Street-level spatial scan statistic and STAC for analysing street crime concentrations. *Transactions in GIS* 15, 3 (2011), 365–383.

Tetsuji Yokoyama Takahashi and Toshiro Tango. 2023. *FleXScan: Software for the Flexible Scan Statistics*. https://sites.google.com/site/flexscansoftware/.

Toshiro Tango and Kunihiko Takahashi. 2005. A flexibly shaped spatial scan statistic for detecting clusters. *International journal of health geographics* 4 (2005), 1–15.

Toshiro Tango and Kunihiko Takahashi. 2012. A Flexible Spatial Scan Statistic with a Restricted Likelihood Ratio for Detecting Clusters. *Statistics in Medicine* 31, 30 (2012), 4207–4218. `https://doi.org/10.1002/sim.5478`

Yiqun Xie, Shashi Shekhar, and Yan Li. 2022. Statistically-robust clustering techniques for mapping spatial hotspots: A survey. *ACM Computing Surveys (CSUR)* 55, 2 (2022), 1–38.

Appendix A Implementation and pyScan

We include these listings as a reproducibility aid: together they reproduce, in about fifty lines of Python, the full Geom 50 pipeline used throughout Section 4 on any input shapefile, so a reader can replicate our results without reconstructing the `pyScan` calls from the body of the paper.

We illustrate how to run `pyScan` on U.S. counties using the Geom 50 setting. First, we load a shapefile of U.S. counties.

```
import pyscan
import geopandas as gpd
import numpy as np
import random
from shapely.geometry import Point, Polygon
import matplotlib.pyplot as plt

# Load shapefile of US counties
us_df = gpd.read_file("us_county.shp").to_crs("EPSG:4326")

# Check the first few entries
print(us_df.head())
```

Listing 1: Python Code to Load Shapefile

Next, we define a target rectangle over the state of interest:

```
# Define target rectangle
target_rectangle = Polygon([(-100, 33), (-100, 40), (-90, 40), (-90, 33)])
```

Listing 2: Defining a Target Rectangle Using Polygon

We generate 50 random points inside each region and assign all of them to the `baseline` set. Then, we construct the `measured` set probabilistically: points inside the target rectangle are included with probability 0.4, and those outside with probability 0.2.

```
# Sample 50 points from each region
n_geom = 50
sampled_points = []

for i, row in us_df.iterrows():
    polygon = row['geometry']
    min_x, min_y, max_x, max_y = polygon.bounds
    region_points = []

    while len(region_points) < n_geom:
        # Generate a random point within the bounding box
        random_point = Point([random.uniform(min_x, max_x),
    random.uniform(min_y, max_y)])
        if random_point.within(polygon):
            region_points.append([random_point.x, random_point.y, i])

    sampled_points.append(region_points)

sampled_points = np.vstack(sampled_points)
```

Listing 3: Python Code to Generate 50 Random Points Inside Each Region

```
# Create baseline and measured sets to test the algorithm using p=0.4 (inside
    target), q=0.2 (outside)
baseline = []
measured = []

for point in sampled_points:
```

```
    prob = random.random()
    # WPoint(weight, x, y, value): defines a weighted point with attribute for
    pyScan
    baseline.append(pyscan.WPoint(1.0, point[0], point[1], 1.0))
    pt = Point(point[0], point[1])

    if target_rectangle.contains(pt):
        if prob <= 0.4:
            measured.append(pyscan.WPoint(1.0, point[0], point[1], 1.0))
    else:
        if prob <= 0.2:
            measured.append(pyscan.WPoint(1.0, point[0], point[1], 1.0))
```

Listing 4: Assigning Points to Baseline and Measured Sets Using Probabilistic Inclusion

We use the `Kulldorff` scan statistic (Poisson model) to detect anomalous regions. Then we pass the `measure` and `baseline` lists to `pyscan.Grid()`. We then apply `pyscan.max_subgrid()` to find the most anomalous rectangular subregion, stored in the variable `subgrid`. This rectangle represents the discovered rectangle.

```
# Run pyScan to find most anomalous rectangle
rect_f = pyscan.KULLDORF
grid = pyscan.Grid(100, measured, baseline)
subgrid = pyscan.max_subgrid(grid, rect_f)
rect = grid.toRectangle(subgrid)
```

Listing 5: Python Code to Run pyScan with Geom 50 Sampling

To visualize the result, we plot the detected rectangle over the sampled points:

```
# Plot discovered rectangle
plt.scatter(sampled_points[:, 0], sampled_points[:, 1], s=2)
plt.gca().add_patch(
    plt.Rectangle((rect.lowX(), rect.lowY()),
                  rect.upX() - rect.lowX(),
                  rect.upY() - rect.lowY(),
                  edgecolor='red', facecolor='none')
)
plt.axis('off')
plt.show()
```

Listing 6: Python Code to Plot the Detected Rectangle Over Sampled Points

Full examples and further documentation for `pyScan` are available at:
`https://mmath.dev/pyscan`

Appendix B Disk Cluster Recovery vs. Buchin

We repeat the head-to-head comparison with the area-based method of Buchin et al. (Buchin et al., 2012) using *circular* planted targets. We plant disks of radius approximately 67 km and 44 km, centered in the state, and follow the same protocol as the rectangular comparison in Section 4.4: $q = 0.2$, Point Jaccard distance averaged over 20 trials with ± 1 standard deviation bands, Buchin given its standard nine-point size grid, and Centroid included as a lower reference.

Figure 14 shows the recovery curves; the pattern mirrors the rectangular case. On the larger disk the Buchin method is close to the correct radius ($1.17\times$ planted); on the smaller disk it oversizes more ($1.25\times$), where its discrete grid adapts less well. Geom 50 recovers the correct radius at both sizes ($1.02\times$ on average), so the separation concentrates on the smaller, harder target. Buchin's residual oversizing occurs under its own nine-point size-selection protocol with the correct scale among the candidates, so it stems from the area-weighted score itself rather than any imposed restriction on the size search. Centroid degenerates under an unbounded disk search on one point per county, confirming its unsuitability as anything but a lower reference.

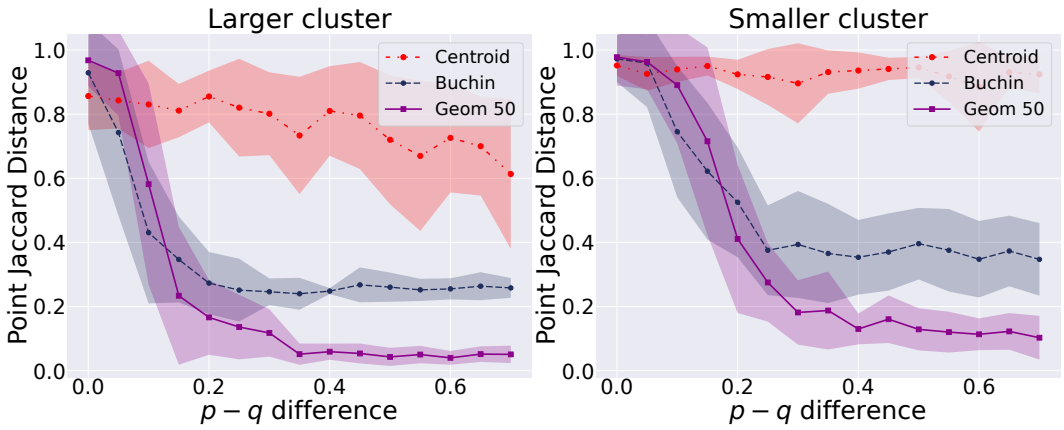

Figure 14: Circular cluster recovery on Arkansas counties: Point Jaccard distance vs. pq difference for planted disks of radius $\sim$67 km (left) and $\sim$44 km (right). Bands show ± 1 standard deviation over 20 trials.

Figure 15 shows the discovered windows geographically. Geom 50 matches (or improves over) the Buchin method's accuracy on easy (large) targets, and visually improves over it on hard (small) targets for both window shapes, and, unlike the area-based method, requires no user-supplied window size. Specifically on the large disk, Centroid degenerates to on average $2.17\times$ the planted radius (offset $\sim$83.8 km), Buchin selects nearly the correct radius at about $1.16\times$ (offset $\sim$7.2 km), and Geom 50 recovers the disk at $1.01\times$ (offset $\sim$1.8 km). On the small disk, Centroid degenerates to $6.57\times$ planted (offset $\sim$ 205.2 km), Buchin oversizes to $1.25\times$ (offset $\sim$ 10.8 km), and Geom 50 stays at $1.03\times$ (offset $\sim$3.6 km).

We note that pyScan's exact disk scan enumerates candidate disks through triples of points and becomes more expensive at a national scale. Note that all windows are defined in EPSG:4326 lon/lat degree-Cartesian coordinates; the resulting geographic ellipticity (1° lon $\approx$ 91 km vs. 1° lat $\approx$ 111 km at this latitude) applies uniformly to all methods. The Buchin method's circular window is rendered as an inscribed regular polygon; our overlap measure treats it as a true circle, a $\sim 2\%$ area difference in its favor. Centroid runs on an independent random stream; comparisons rely on mean-over-trials equivalence, not paired trials.

Appendix C Further Discussion

This paper uses a model where the data in each region z is uniformly mapped to the area in that region through a sample. However, the human population is certainly not distributed uniformly in spatial regions, and one could think of other ways to distribute the measured and baseline values. This could make models more realistic in baseline values but perhaps less realistic in measured values. Our view in this paper is that when data are aggregated to regions, this fine-grained spatial information is lost, and uniform is a reasonable approach, but future work may address this more carefully. Moreover, Section 4.5 evaluated a population-weighted alternative ("Weighted Sampling"), which did not outperform uniform sampling. This sensitivity check does not isolate population weighting at a fixed point budget and does not model nonuniform population density within counties; both remain directions for future work.

There are also other ways we could assess statistical power. This could involve how we evaluate similar rectangles, how we distribute our $50n$ samples, or experiments other than changing the pq difference or the target region size. We informally explored other ideas beyond the ones reported in this paper, and they either gave results similar to the ones we showed or were far less reliable in measurement. The paper presents the ones we feel most clearly demonstrate the merits of the main conceptual frameworks.

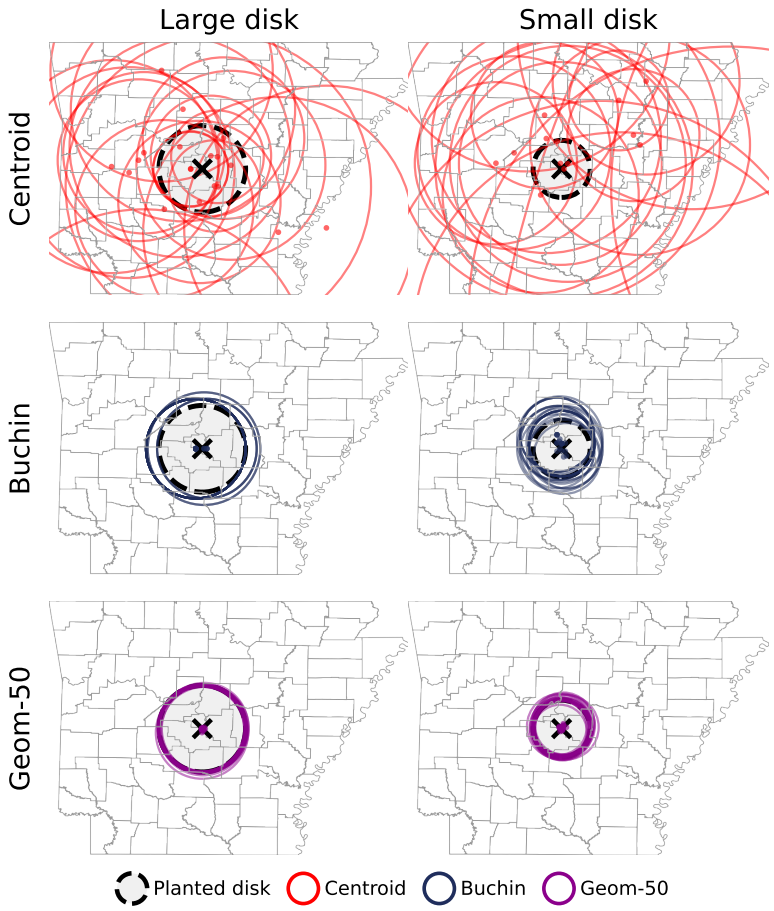

Figure 15: Discovered disk windows on Arkansas for both planted disk targets (black dashed) at $p - q = 0.5$, 20 trials each, planted center $(-92.5,\ 34.75)$. **Columns:** large disk (radius $\sim$67 km) and small disk (radius $\sim$44 km).

Generative AI Statement: ChatGPT and Claude were utilized to generate some text suggestions, help with formatting, refine figures, update and compile some code, and facilitate some evaluations. All results were reviewed, verified, and approved by the authors.